

Design Patterns — First-Principles Notes

with UML structure diagrams, real-life analogies, and C++ code examples

Each pattern below is explained the same way: the underlying **problem** (from first principles), the **solution** (how the pattern restructures code to fix it), a text-based **UML structure diagram**, a **C++ code example**, and a **real-life analogy**.

1. Memento Design Pattern

The Problem

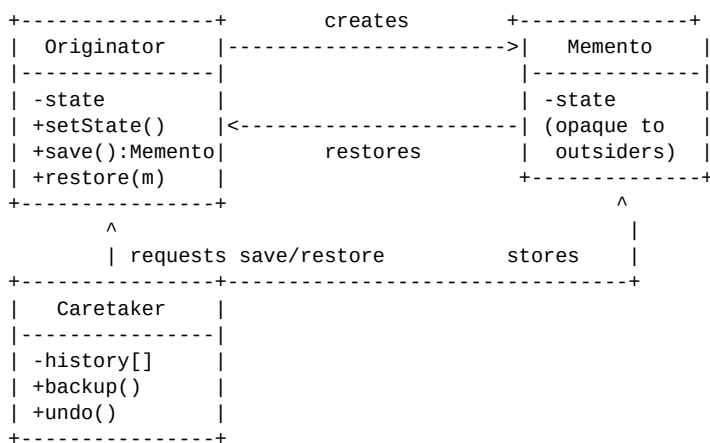
An object (e.g. a Text Editor) has internal state that changes over time. We want to **undo** changes and restore a previous state. Naively exposing all internal fields publicly so an external History Manager can read/restore them **breaks encapsulation** — any external class could then corrupt the object's state directly.

Core question: How do you capture and restore an object's internal state without exposing its internals?

The Solution

Three roles: **Originator** (creates/restores snapshots of itself), **Memento** (an opaque snapshot — a sealed envelope only the Originator can read), and **Caretaker** (stores a history of Mementos for undo/redo, without looking inside them).

UML Structure



C++ Code Example

```
#include <iostream>
#include <string>
#include <vector>

// ---- Memento: opaque snapshot ----
class Memento {
    std::string state;
public:
    explicit Memento(std::string s) : state(std::move(s)) {}
    std::string getState() const { return state; } // only Originator truly "understands" it
};

// ---- Originator ----
class Editor {
    std::string text;
public:
    void type(const std::string& s) { text += s; }
    std::string getText() const { return text; }

    Memento save() const { return Memento(text); }
    void restore(const Memento& m) { text = m.getState(); }
};

// ---- Caretaker ----
class History {
    std::vector<Memento> history;
public:
    void backup(const Editor& e) { history.push_back(e.save()); }
    void undo(Editor& e) {
```

```

        if (history.empty()) return;
        e.restore(history.back());
        history.pop_back();
    }
};

int main() {
    Editor editor;
    History history;

    editor.type("Hello");
    history.backup(editor);        // save checkpoint

    editor.type(", World!");
    std::cout << editor.getText() << "\n";    // Hello, World!

    history.undo(editor);
    std::cout << editor.getText() << "\n";    // Hello
}

```

Real-Life Analogy: Ctrl+Z in a word processor, or a video game 'Save Point'. You (the Caretaker) don't understand or edit the raw bytes in the save file — you just choose when to save/load; the engine (Originator) knows how to interpret it.

2. Observer Design Pattern

The Problem

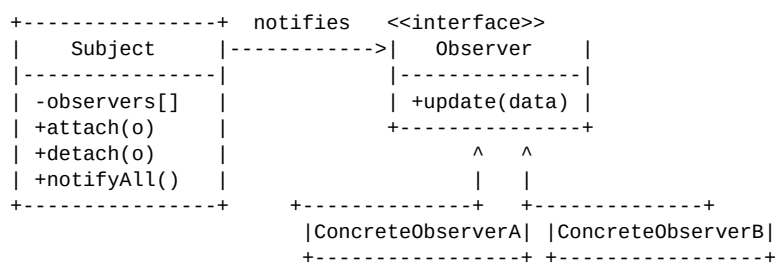
One object's state change must notify many dependent objects automatically (e.g. a stock price update refreshing a chart, ticker, and mobile app). Naively, the Subject calls each dependent directly — **tightly coupling** it to every concrete observer, so adding a new observer means editing the Subject.

Core question: How does one object broadcast changes to an open-ended, dynamic set of listeners without knowing their concrete types?

The Solution

Define a common **Observer** interface with *update()*. The **Subject** keeps a list of registered observers (typed only as the interface) and loops through calling *update()* on each. Observers can subscribe/unsubscribe at runtime.

UML Structure



C++ Code Example

```
#include <iostream>
#include <vector>
#include <string>

// ---- Observer interface ----
class Observer {
public:
    virtual void update(const std::string& msg) = 0;
    virtual ~Observer() = default;
};

// ---- Subject ----
class Channel {
    std::vector<Observer*> subscribers;
public:
    void attach(Observer* o) { subscribers.push_back(o); }
    void notifyAll(const std::string& video) {
        for (auto* o : subscribers) o->update(video);
    }
};

// ---- Concrete Observers ----
class Subscriber : public Observer {
    std::string name;
public:
    explicit Subscriber(std::string n) : name(std::move(n)) {}
    void update(const std::string& msg) override {
        std::cout << name << " notified: New video -> " << msg << "\n";
    }
};

int main() {
    Channel channel;
    Subscriber alice("Alice"), bob("Bob");

    channel.attach(&alice);
}
```

```
channel.attach(&bob);  
channel.notifyAll("Design Patterns Explained");  
}
```

Real-Life Analogy: YouTube channel subscriptions. The channel doesn't know or care who its subscribers are individually — it just notifies 'all subscribers' when it uploads.

3. Command Pattern

The Problem

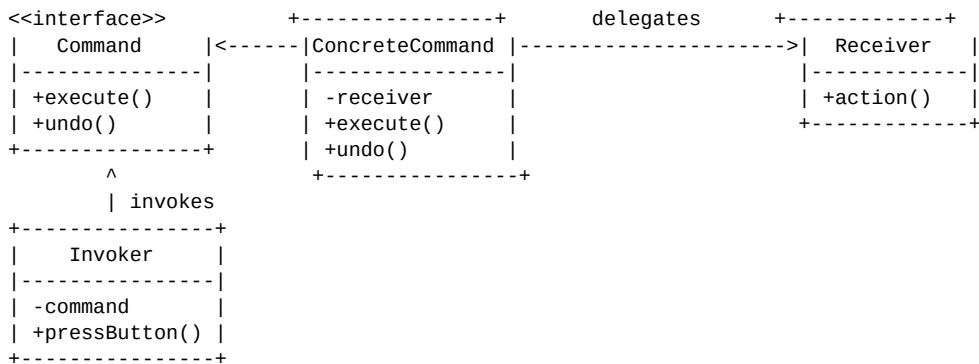
You want to decouple the object that **invokes** an action (a button, menu item) from the object that **performs** it (a document, device). Naively, the invoker hardcodes if/else logic for each receiver's method — tightly coupling it, and making undo, queuing, or logging of actions hard since the action is just a raw call, not an object.

Core question: How do you turn a request into an object so it can be passed around, queued, logged, or undone?

The Solution

Encapsulate a request as a **Command** object with *execute()* (and optionally *undo()*). The **Invoker** only knows the Command interface. The Command holds a reference to the **Receiver** and calls the right method on it.

UML Structure



C++ Code Example

```
#include <iostream>
#include <memory>
#include <string>

// ---- Receiver ----
class Light {
public:
    void on() { std::cout << "Light is ON\n"; }
    void off() { std::cout << "Light is OFF\n"; }
};

// ---- Command interface ----
class Command {
public:
    virtual void execute() = 0;
    virtual void undo() = 0;
    virtual ~Command() = default;
};

// ---- Concrete Commands ----
class LightOnCommand : public Command {
    Light& light;
public:
    explicit LightOnCommand(Light& l) : light(l) {}
    void execute() override { light.on(); }
    void undo() override { light.off(); }
};

// ---- Invoker ----
class RemoteButton {
    std::unique_ptr<Command> command;
public:
    void setCommand(std::unique_ptr<Command> c) { command = std::move(c); }
    void press() { if (command) command->execute(); }
```

```
void pressUndo() { if (command) command->undo(); }  
};  
  
int main() {  
    Light light;  
    RemoteButton button;  
    button.setCommand(std::make_unique<LightOnCommand>(light));  
  
    button.press();      // Light is ON  
    button.pressUndo();  // Light is OFF  
}
```

Real-Life Analogy: A restaurant order slip. The waiter writes the order (Command) and hands it to the kitchen (Receiver) without needing to know how the dish is cooked.

4. Template Design Pattern (Template Method)

The Problem

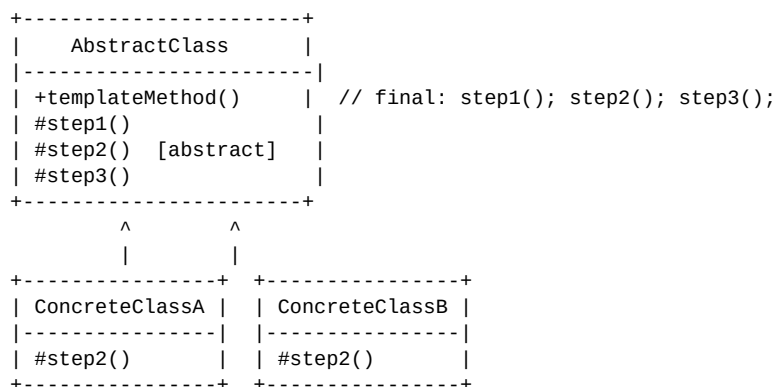
Several classes follow the **same overall algorithm**, but a few steps differ. Naively copy-pasting the whole algorithm into every subclass duplicates code — a bug fix in shared logic must then be repeated everywhere.

Core question: How do you share a fixed algorithm's skeleton while letting subclasses customize only the steps that vary?

The Solution

Define an abstract base class with a non-overridable *templateMethod()* that calls a fixed sequence of steps. Some steps are implemented in the base class (shared); others are abstract 'hooks' that subclasses override.

UML Structure



C++ Code Example

```
#include <iostream>

// ---- Abstract Class ----
class Beverage {
public:
    // Template method - fixed algorithm skeleton
    void prepare() {
        boilWater();
        brew();      // varies
        pourInCup();
        addCondiments(); // varies
    }
    virtual ~Beverage() = default;

protected:
    void boilWater() { std::cout << "Boiling water\n"; }
    void pourInCup() { std::cout << "Pouring into cup\n"; }

    virtual void brew() = 0;
    virtual void addCondiments() = 0;
};

// ---- Concrete Classes ----
class Tea : public Beverage {
protected:
    void brew() override { std::cout << "Steeping tea bag\n"; }
    void addCondiments() override { std::cout << "Adding lemon\n"; }
};

class Coffee : public Beverage {
protected:
    void brew() override { std::cout << "Dripping coffee through filter\n"; }
    void addCondiments() override { std::cout << "Adding milk and sugar\n"; }
```



```
};  
  
int main() {  
    Tea tea;  
    Coffee coffee;  
  
    tea.prepare();  
    std::cout << "---\n";  
    coffee.prepare();  
}
```

Real-Life Analogy: A recipe template for tea/coffee: boil water -> brew -> pour into cup -> add condiments. 'Brew' and 'condiments' differ, but the sequence and shared steps never change.

5. Iterator Design Pattern

The Problem

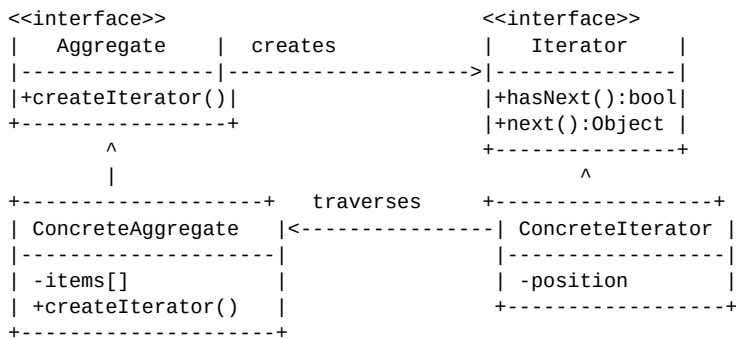
A collection has its own internal storage structure. If client code must know that structure to traverse it (array indices vs. linked-list pointers), the client becomes **coupled** to the internal representation, and changing it breaks all client code.

Core question: How do you traverse a collection sequentially without exposing its underlying representation?

The Solution

Extract traversal into a separate **Iterator** object with *hasNext()* / *next()*. The collection exposes *createIterator()*; the client only ever talks to the Iterator interface.

UML Structure



C++ Code Example

```
#include <iostream>
#include <vector>
#include <string>
#include <memory>

// ---- Iterator interface ----
class Iterator {
public:
    virtual bool hasNext() = 0;
    virtual std::string next() = 0;
    virtual ~Iterator() = default;
};

// ---- Concrete Aggregate + Iterator ----
class PlaylistIterator : public Iterator {
    const std::vector<std::string>& songs;
    size_t pos = 0;
public:
    explicit PlaylistIterator(const std::vector<std::string>& s) : songs(s) {}
    bool hasNext() override { return pos < songs.size(); }
    std::string next() override { return songs[pos++]; }
};

class Playlist {
    std::vector<std::string> songs;
public:
    void add(const std::string& s) { songs.push_back(s); }
    std::unique_ptr<Iterator> createIterator() const {
        return std::make_unique<PlaylistIterator>(songs);
    }
};

int main() {
    Playlist playlist;
    playlist.add("Song A");
```

```
playlist.add("Song B");
playlist.add("Song C");

auto it = playlist.createIterator();
while (it->hasNext()) {
    std::cout << "Now playing: " << it->next() << "\n";
}
}
```

Real-Life Analogy: A TV remote's channel up/down buttons. You don't need to know how channels are stored internally — you just press 'next' repeatedly.

6. State Design Pattern

The Problem

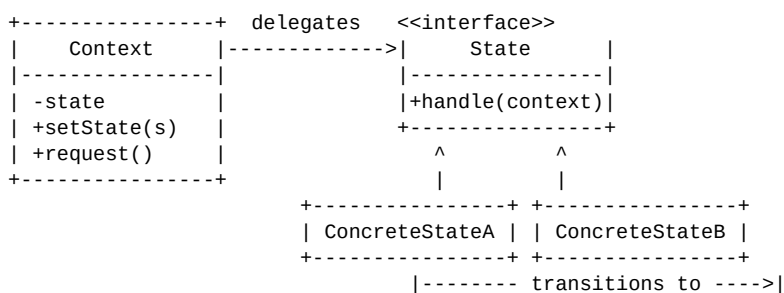
An object's behavior must change based on internal state (Traffic Light: Red/Yellow/Green). The naive fix is a giant if/switch on a state variable in every method — creating fragile, scattered conditionals; adding a state means editing every switch.

Core question: How do you let an object change behavior when its state changes, as if it changed its class, without giant conditionals?

The Solution

Represent each state as a class implementing a common **State** interface. The **Context** holds a State reference and delegates behavior to it. Transitioning state = swapping which State object the Context points to.

UML Structure



C++ Code Example

```
#include <iostream>
#include <memory>

class TrafficLightContext;

// ---- State interface ----
class State {
public:
    virtual void handle(TrafficLightContext& ctx) = 0;
    virtual ~State() = default;
};

// ---- Context ----
class TrafficLightContext {
    std::unique_ptr<State> state;
public:
    void setState(std::unique_ptr<State> s) { state = std::move(s); }
    void request() { state->handle(*this); }
};

// ---- Concrete States ----
class GreenState : public State {
public:
    void handle(TrafficLightContext& ctx) override;
};

class YellowState : public State {
public:
    void handle(TrafficLightContext& ctx) override;
};

class RedState : public State {
public:
    void handle(TrafficLightContext& ctx) override {
        std::cout << "RED -> STOP. Switching to GREEN.\n";
        ctx.setState(std::make_unique<GreenState>());
    }
}
```

```

};
void GreenState::handle(TrafficLightContext& ctx) {
    std::cout << "GREEN -> GO. Switching to YELLOW.\n";
    ctx.setState(std::make_unique<YellowState>());
}
void YellowState::handle(TrafficLightContext& ctx) {
    std::cout << "YELLOW -> CAUTION. Switching to RED.\n";
    ctx.setState(std::make_unique<RedState>());
}

int main() {
    TrafficLightContext light;
    light.setState(std::make_unique<RedState>());

    for (int i = 0; i < 4; ++i) light.request();
}

```

Real-Life Analogy: A traffic light. Instead of one controller checking 'if color==red...else if...' everywhere, each color is its own self-contained behavior that transitions to the next.

7. Mediator Design Pattern

The Problem

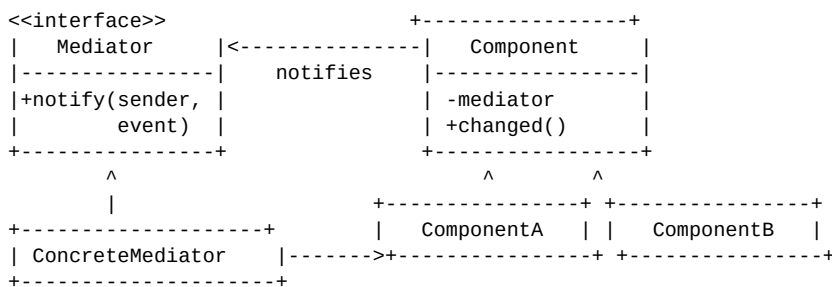
Many objects need to talk to each other (UI widgets referencing one another). Naively, each object holds direct references to every other object — a tangled many-to-many web where changing one object risks breaking many others.

Core question: How do you reduce chaotic many-to-many communication into structured one-to-many communication?

The Solution

Introduce a **Mediator** that all components talk to instead of each other. Each component only knows the Mediator; the Mediator decides who to inform and how.

UML Structure



C++ Code Example

```
#include <iostream>
#include <string>

class Mediator {
public:
    virtual void notify(const std::string& sender, const std::string& event) = 0;
    virtual ~Mediator() = default;
};

class Button; class TextBox; // fwd decl

class DialogMediator : public Mediator {
    TextBox* textBox = nullptr;
public:
    void registerTextBox(TextBox* t) { textBox = t; }
    void notify(const std::string& sender, const std::string& event) override;
};

class TextBox {
public:
    void showMessage(const std::string& msg) { std::cout << "TextBox displays: " << msg << "\n"; }
};

class Button {
    Mediator& mediator;
public:
    explicit Button(Mediator& m) : mediator(m) {}
    void click() { mediator.notify("Button", "clicked"); }
};

void DialogMediator::notify(const std::string& sender, const std::string& event) {
    if (sender == "Button" && event == "clicked" && textBox)
        textBox->showMessage("Button was clicked!");
}

int main() {
```

```
DialogMediator mediator;  
TextBox textBox;  
Button button(mediator);  
  
mediator.registerTextBox(&textBox);  
button.click(); // TextBox displays: Button was clicked!  
}
```

Real-Life Analogy: *An air traffic control tower. Pilots don't talk directly to every other plane — they talk only to the tower, which coordinates all planes.*

8. Visitor Design Pattern

The Problem

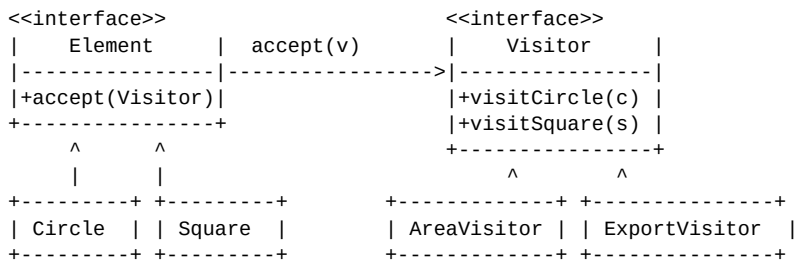
A stable set of element types (Circle, Square) keeps needing **new operations** (area, export, render). Adding a method to every element class each time pollutes them with unrelated logic and violates Open/Closed Principle.

Core question: How do you add new operations to classes without modifying those classes each time?

The Solution

Move each operation into a **Visitor** class with *visit(ElementType)* per type. Each **Element** only needs *accept(Visitor)*, calling back *visitor.visit(this)* — double dispatch. New operations = new Visitor classes, zero changes to Elements.

UML Structure



C++ Code Example

```
#include <iostream>

class Circle; class Square;

// ---- Visitor interface ----
class Visitor {
public:
    virtual void visit(Circle& c) = 0;
    virtual void visit(Square& s) = 0;
    virtual ~Visitor() = default;
};

// ---- Element interface ----
class Shape {
public:
    virtual void accept(Visitor& v) = 0;
    virtual ~Shape() = default;
};

class Circle : public Shape {
public:
    double radius = 5.0;
    void accept(Visitor& v) override { v.visit(*this); }
};

class Square : public Shape {
public:
    double side = 4.0;
    void accept(Visitor& v) override { v.visit(*this); }
};

// ---- Concrete Visitor: computes area ----
class AreaVisitor : public Visitor {
public:
    void visit(Circle& c) override {
        std::cout << "Circle area: " << 3.14159 * c.radius * c.radius << "\n";
    }
    void visit(Square& s) override {
```



```
        std::cout << "Square area: " << s.side * s.side << "\n";
    }
};

int main() {
    Circle circle;
    Square square;
    AreaVisitor areaVisitor;

    circle.accept(areaVisitor); // Circle area: 78.5397
    square.accept(areaVisitor); // Square area: 16
}
```

Real-Life Analogy: A tax auditor visiting different business types (restaurant, factory, shop). Each business doesn't change; a new type of auditor (Visitor) can be added later without touching them.

9. Strategy Design Pattern

The Problem

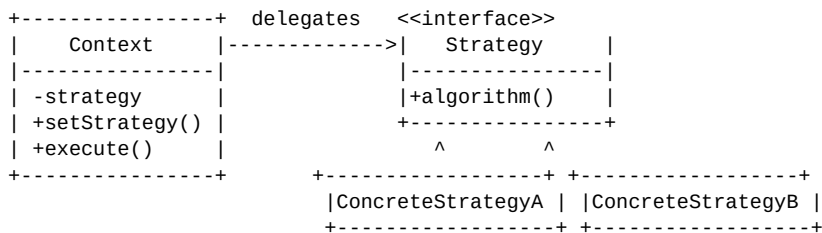
A class must perform a task in multiple interchangeable ways (sorting, route calculation). Naive if/else blocks choosing behavior inline bloat the class; adding a new variant means modifying it.

Core question: How do you let an algorithm vary independently of clients, and swap it at runtime?

The Solution

Extract each algorithm into its own class implementing a common **Strategy** interface. The **Context** holds a Strategy reference and delegates the work to it.

UML Structure



C++ Code Example

```
#include <iostream>
#include <memory>
#include <vector>
#include <algorithm>

// ---- Strategy interface ----
class SortStrategy {
public:
    virtual void sort(std::vector<int>& data) = 0;
    virtual ~SortStrategy() = default;
};

// ---- Concrete Strategies ----
class AscendingSort : public SortStrategy {
public:
    void sort(std::vector<int>& data) override {
        std::sort(data.begin(), data.end());
    }
};
class DescendingSort : public SortStrategy {
public:
    void sort(std::vector<int>& data) override {
        std::sort(data.begin(), data.end(), std::greater<int>());
    }
};

// ---- Context ----
class Sorter {
    std::unique_ptr<SortStrategy> strategy;
public:
    void setStrategy(std::unique_ptr<SortStrategy> s) { strategy = std::move(s); }
    void execute(std::vector<int>& data) { strategy->sort(data); }
};

int main() {
    std::vector<int> nums = {5, 2, 8, 1, 9};
    Sorter sorter;

    sorter.setStrategy(std::make_unique<AscendingSort>());
    sorter.execute(nums);
}
```

```
for (int n : nums) std::cout << n << " "; // 1 2 5 8 9
std::cout << "\n";

sorter.setStrategy(std::make_unique<DescendingSort>());
sorter.execute(nums);
for (int n : nums) std::cout << n << " "; // 9 8 5 2 1
}
```

Real-Life Analogy: Google Maps' travel-mode selector: Driving, Walking, Cycling, Transit are interchangeable strategies the app uses to compute your route.

10. Chain of Responsibility

The Problem

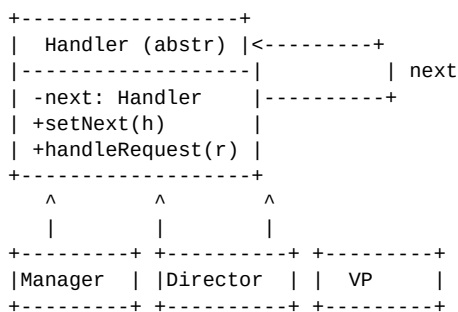
A request may need handling by one of several possible handlers (expense approval by Manager -> Director -> VP based on amount), but the sender shouldn't need to know which one handles it. A big conditional in the sender couples it to every handler and condition.

Core question: How do you let a request pass along handlers until one handles it, without the sender knowing which one will?

The Solution

Each **Handler** implements a common interface, holds a reference to the **next** handler, and decides: handle it, or pass to next. The sender sends the request to the first handler only.

UML Structure



C++ Code Example

```
#include <iostream>
#include <memory>

// ---- Handler (abstract) ----
class Approver {
protected:
    std::shared_ptr<Approver> next;
public:
    void setNext(std::shared_ptr<Approver> n) { next = std::move(n); }
    virtual void handleRequest(double amount) {
        if (next) next->handleRequest(amount);
        else std::cout << "No one could approve $" << amount << "\n";
    }
    virtual ~Approver() = default;
};

// ---- Concrete Handlers ----
class Manager : public Approver {
public:
    void handleRequest(double amount) override {
        if (amount <= 1000) std::cout << "Manager approved $" << amount << "\n";
        else Approver::handleRequest(amount);
    }
};

class Director : public Approver {
public:
    void handleRequest(double amount) override {
        if (amount <= 5000) std::cout << "Director approved $" << amount << "\n";
        else Approver::handleRequest(amount);
    }
};

class VP : public Approver {
public:
    void handleRequest(double amount) override {
```

```

        std::cout << "VP approved $" << amount << "\n";
    }
};

int main() {
    auto manager = std::make_shared<Manager>();
    auto director = std::make_shared<Director>();
    auto vp = std::make_shared<VP>();

    manager->setNext(director);
    director->setNext(vp);

    manager->handleRequest(500);    // Manager approved $500
    manager->handleRequest(3000);   // Director approved $3000
    manager->handleRequest(9000);   // VP approved $9000
}

```

Real-Life Analogy: Customer support escalation: Level-1 -> Level-2 -> Specialist. You don't decide who resolves your issue — each level decides to handle it or forward it.

11. Null Object Pattern & Anti-Patterns

11a. Null Object Pattern — The Problem

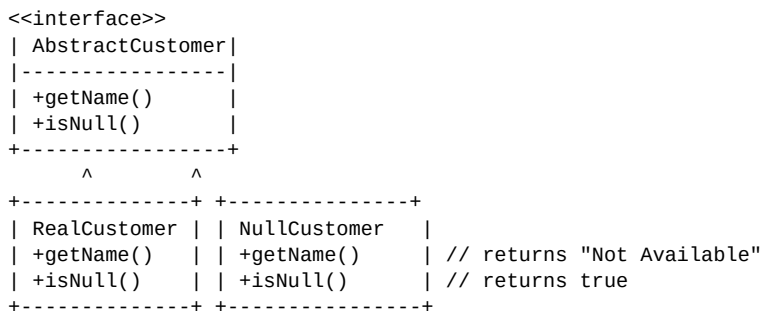
Client code frequently checks *if (object != nullptr)* before calling a method, to avoid a crash. Scattering these null-checks everywhere is repetitive, error-prone (easy to forget one), and clutters business logic.

Core question: How do you avoid null-reference errors and repetitive checks while keeping graceful 'do nothing' behavior?

The Solution

Return a special **Null Object** implementing the same interface as a real object, but with safe no-op methods (empty/default values). Client code calls it exactly like a real object — no special-casing needed.

UML Structure



C++ Code Example

```
#include <iostream>
#include <string>
#include <memory>

// ---- Common interface ----
class Customer {
public:
    virtual std::string getName() const = 0;
    virtual bool isNull() const { return false; }
    virtual ~Customer() = default;
};

// ---- Real Object ----
class RealCustomer : public Customer {
    std::string name;
public:
    explicit RealCustomer(std::string n) : name(std::move(n)) {}
    std::string getName() const override { return name; }
};

// ---- Null Object ----
class NullCustomer : public Customer {
public:
    std::string getName() const override { return "Not Available"; }
    bool isNull() const override { return true; }
};

// ---- Factory / Lookup ----
std::unique_ptr<Customer> findCustomer(const std::string& name) {
    if (name == "John") return std::make_unique<RealCustomer>("John");
    return std::make_unique<NullCustomer>(); // instead of nullptr
}

int main() {
    auto c1 = findCustomer("John");
    auto c2 = findCustomer("Unknown");
```

```

// No null-checks needed anywhere!
std::cout << c1->getName() << "\n"; // John
std::cout << c2->getName() << "\n"; // Not Available
}

```

Real-Life Analogy: A voicemail/answering machine. When someone isn't available, you don't get silence or an error — you get a safe stand-in that behaves like a normal call but does nothing further.

11b. Anti-Patterns — The Concept

A design pattern is a proven *good* solution to a recurring problem. An **anti-pattern** is the mirror image: a commonly repeated *bad* solution that looks convenient short-term but leads to fragile, unmaintainable systems long-term. They are documented so they can be recognized and avoided.

Anti-Pattern	The Problem It Represents
God Object / God Class	One class does everything — impossible to test, reuse, or reason about. Violates Single Responsibility.
Spaghetti Code	Tangled control flow with no clear structure — nearly impossible to trace or modify safely.
Copy-Paste Programming	Duplicated logic instead of abstraction (Template Method/Strategy) — bug fixes must be repeated everywhere.
Golden Hammer	Over-relying on one familiar tool/pattern for every problem, even where it doesn't fit.
Magic Numbers/Strings	Hardcoded literals scattered through code with no named meaning — brittle and unclear.
Lava Flow	Dead/obsolete code left in a codebase because nobody understands it well enough to remove it.
Boat Anchor	Keeping unused components 'just in case,' adding weight and confusion with no benefit.

C++ Sketch: God Object vs. Proper Separation

```

// --- ANTI-PATTERN: God Object doing everything ---
class GodObject {
public:
    void validateUser();
    void connectToDatabase();
    void renderUI();
    void calculateTax();
    void sendEmail();
    void logActivity();
    void generateReport();
    // ... one class, every unrelated responsibility
};

// --- CORRECT: separated, single-responsibility classes ---
class UserValidator { public: void validate(); };
class DatabaseConnector { public: void connect(); };
class UIRenderer { public: void render(); };
class TaxCalculator { public: void calculate(); };
class EmailService { public: void send(); };
class Logger { public: void log(); };
class ReportGenerator { public: void generate(); };

```

Real-Life Analogy: A Swiss Army knife used as your only tool for building a house. Convenient for one task, but using it for everything yields a poorly built house. A proper toolbox with dedicated tools (like well-separated classes) is what actually scales.

Quick Comparison Table

Pattern	Category	Solves
Memento	Behavioral	Save/restore state without breaking encapsulation
Observer	Behavioral	One-to-many automatic notification
Command	Behavioral	Decouple invoker from receiver; enables undo/queueing
Template Method	Behavioral	Share algorithm skeleton, vary only specific steps
Iterator	Behavioral	Traverse a collection without exposing its structure
State	Behavioral	Change behavior based on internal state, avoid big conditionals
Mediator	Behavioral	Centralize chaotic many-to-many communication
Visitor	Behavioral	Add new operations without modifying existing classes
Strategy	Behavioral	Swap interchangeable algorithms at runtime
Chain of Responsibility	Behavioral	Pass a request along handlers until one processes it
Null Object	Behavioral/Structural	Avoid null-checks with a safe no-op stand-in object
Anti-Patterns	(meta)	Recognize/avoid common bad recurring solutions